Assignments 01

1. Flowchart

[10 points] Write a function Print_values with arguments a, b, and c to reflect the following flowchart. Here the purple parallelogram operator is to print values in the given order. Report your output with some random a, b, and c values.

In [1]:
```
from tracemalloc import stop


def Print_values(a,b,c):
    if a > b :
        if b > c:
            print(a,b,c)
        else:
            if a > c:
                print(a,c,b)
            else:
                print(c,a,b)
    else:
        if b > c:
            pass
        else:
            print(c,b,a)
import random
for i in range(3):
    a = random.randint(0, 100)
    b = random.randint(0, 100)
    c = random.randint(0, 100)
    print(f"a={a}, b={b}, c={c}")
    Print_values(a, b, c)
```

```
a=37, b=52, c=66
66 52 37
a=31, b=69, c=86
86 69 31
a=31, b=7, c=88
88 31 7
```

2. Matrix multiplication

2.1 [5 points] Make two matrices M1 (5 rows and 10 columns) and M2 (10 rows and 5 columns); both are filled with random integers from 0 and 50

In [6]:
```
import random

M1 = [[random.randint(0, 51) for j in range(10)] for i in range(5)]   # 5行10列
M2 = [[random.randint(0, 51) for j in range(5)] for i in range(10)]   # 10行5列
for row in M1:
```

```
    print(row)
print(" ")
for row in M2:
    print(row)
```

```
[7, 23, 16, 5, 2, 15, 13, 34, 42, 38]
[2, 2, 51, 42, 46, 34, 21, 25, 40, 8]
[42, 15, 23, 48, 21, 4, 29, 31, 13, 48]
[32, 7, 32, 33, 10, 46, 32, 17, 43, 24]
[31, 34, 41, 0, 1, 8, 19, 1, 25, 13]

[48, 0, 16, 18, 5]
[44, 25, 27, 48, 24]
[34, 5, 14, 0, 4]
[8, 19, 35, 26, 48]
[12, 38, 19, 26, 17]
[24, 49, 51, 22, 32]
[4, 27, 24, 48, 8]
[8, 32, 14, 41, 35]
[50, 34, 9, 26, 14]
[6, 41, 3, 20, 22]
```

2.2 [10 points] Write a function Matrix_multip to do matrix multiplication, i.e., M1 * M2. Here you are ONLY allowed to use for loop, * operator, and + operator

In [8]:
```
def Matrix_multip(M1,M2):
    rows = len(M1)
    cols = len(M2[0])
    n = len(M2)
    result = [[0 for j in range(cols)] for i in range(rows)]
    for i in range(rows):
        for j in range(cols):
            for k in range(n):
                result[i][j] += M1[i][k] * M2[k][j]
    return result
M3 = Matrix_multip(M1,M2)
for row in M3:
    print(row)
```

```
[4968, 5986, 3215, 5612, 4123]
[5954, 7572, 6116, 6401, 5927]
[5492, 6581, 5073, 7319, 6006]
[6978, 7450, 6305, 6873, 5663]
[5994, 3413, 3149, 4255, 2231]
```

3. Pascal triangle

[20 points] One of the most interesting number patterns is Pascal's triangle (named after Blaise Pascal). Write a function Pascal_triangle with an argument k to print the kth line of the Pascal triangle. Report Pascal_triangle(100) and Pascal_triangle(200).

In [5]:
```
def Pascal_triangle(k):
    row = [1]
    for i in range(k-1):
```

```
        new_row = [1]
        for j in range(len(row)-1):
            new_row.append(row[j] + row[j + 1])
        new_row.append(1)
        row = new_row
    return row
print(Pascal_triangle(100))
print(Pascal_triangle(200))
```

[1, 99, 4851, 156849, 3764376, 71523144, 1120529256, 14887031544, 171200862756, 1731
030945644, 15579278510796, 126050526132804, 924370524973896, 6186171974825304, 38000
770702498296, 215337700647490344, 1130522928399324306, 5519611944537877494, 25144898
858450330806, 107196674080761936594, 428786696323047746376, 161305471473908437922 4,
571901217043857188997 6, 191462581358160885012 24, 606298174300842802538 76, 1818894522
90252840761628, 517685364210719623706172, 139966783656972342705 7428, 359914586546500
3098147672, 881170194648328344718 9128, 205606378751276613767746 32, 45764000431735762
419272568, 972485009174384951409 54207, 197443926105102399225573693, 3832735036157870
10261407757, 711793649572175876199757263, 126541093257275711324401 2912, 215461861492
1181030658724688, 351543037171350589212739 2912, 549849365832112460050 6947888, 824774
048748168690076042 1832, 118686997258882811149874753368, 163901091452742930164937070 3
2, 217264237507124349288404953 68, 276518120463612808185242668 32, 337966591677748987
8196326128, 396743390230400985657087306 72, 447391482600239409357992069 28, 4846741061
502593601378247417 2, 504456722727820966674062486 28, 504456722727820966674062486 28, 4
846741061502593601378247417 2, 447391482600239409357992069 28, 396743390230400985657087
30672, 337966591677748987781963261 28, 276518120463612808185242668 32, 21726423750712
434928840495368, 163901091452742930164937070 32, 118686997258882811149874753368, 82477
404874816868900760421832, 549849365832112460050694788 8, 351543037171350589212739 2912,
215461861492118103065872 4688, 126541093257275711324401 2912, 711793649572175876199757
263, 383273503615787010261407757, 197443926105102399225573693, 972485009174384951409
54207, 457640004317357624192725 68, 205606378751276613767746 32, 881170194648328344718
9128, 359914586546500309814767 2, 139966783656972342705 7428, 517685364210719623706617
2, 181889452290252840761628, 606298174300842802538 76, 191462581358160885012 24, 57190
121704385718899 76, 161305471473908437922 4, 428786696323047746376, 107196674080761936
594, 25144898858450330806, 5519611944537877494, 1130522928399324306, 215337700647490
344, 38000770702498296, 6186171974825304, 924370524973896, 126050526132804, 15579278
510796, 1731030945644, 171200862756, 14887031544, 1120529256, 71523144, 3764376, 156
849, 4851, 99, 1]
[1, 199, 19701, 1293699, 63391251, 2472258789, 79936367511, 2203959847089, 528950363
30136, 1122550215450664, 21328454093562616, 366461620334848584, 5741232051912627816,
82585414900589338584, 1097206226536401212616, 13532210127282281622264, 1556204164637
46238656036, 1675208012521503627885564, 1693821434882853668195 4036, 1613587787967350
07338614764, 1452229009170615066047532876, 1237852345912095699154801 8324, 1001535079
87433197477070330076, 7707465614685076505149628192324, 5652141450769056101097273410376,
39564990155383392707680913872632, 2647810879629503973514030389937 68, 169656030435
5200694140471323923032, 1042172758389623283543432384695576 8, 614522557533191660296299
78545842632, 348229449268808607501236545093108248, 1898412158917053376377708907120493352, 9966663834314530225982971762382590098, 5043735940395534993148958437326947110
2, 246252990031076120253743264881256829498, 1160906953003644566910503963011639339062, 5288576119238825249258962498164134766838, 2329832182259266258457326722164199910796262, 9932442461210556154475971815542115409 1838, 4100315990397178309924696057185334822
76562, 164012639615887132396987842287413392910 6248, 6360490170469769280761235835048470603119352, 2392755826033865586572083956994424655459 1848, 8736341039239927839344585
6104215039745835352, 3097430004821428961222171261876715045534162 48, 1066892557216269
975532081212424201849017322632, 357177073502838209199870666768102358149277576 8, 1162
725366934771191650642808840843846741265303 2, 368196366196010877356036889466267218134
73401268, 113464594480811515266860347570217040690499665132, 340393783442434545800581
042710651122071498995396, 994483798684759751456599516938961121346144123804, 28304538
885643162156841678559031970376774871215 96, 785050418148970723972778631731641442525642654480 4, 212254372314351343888936444875591945571747828803 96, 559579708828744452070 8
324455811060383255170032104 4, 143891925127391430532499771720855838426561515111256, 3
609920226880170976517099536154804367543560817703 44, 883808055546524618388669196782727965846871786403256, 211215145478067747784410774146380751160015121835354 4, 492835339
448824744830291806341555086040035284282493 6, 112301823251453507428541903412255995015
68017133650264, 249962122720977161695786817272440763099417155555441 36, 5435684255995
852563860760947035616594384150843031026 4, 115508290439911866982041170124506852630663

205414409311, 23990183399058618527039319948936038623291588816838856 9, 48707342052634
16488823134656299135114425868032509707 31, 96687708850751401942309986460863428390841 8
579587747869, 18768790541616448612330762077697018452339890074350399 81, 3563350088335
87647567439106112798466269061681121724981 9, 66176501640523420262524405420948286592 82
574077974892521, 12023617903700734104036124365214547845738761352940297679, 213753207
17690193962730887760381418392424464627449418096, 37187201796529515524203051309156714
189560369968302412304, 63318749004901607514183573850726297133575765081163566896, 105
53124834150267919030595641787716188929294180193927816 0, 17218256308350437131049919 20
50220632556214799782111453840, 27504487349702646326222598210619659486252493991168453
0160, 43019839187996446817937910021738441760548772652853221 3840, 6589114609807050715
15251533244348285193215378606992378160, 98836719147105760727287729986652242778982306
7910488567240, 14520456269759982131539802306681008507035672232 6520240760, 208952907
29654608433191422831565353705246455163503583952 40, 29454807414091435984137306883049 9
56427877533182288184607 60, 40675686428983411597141995219449939828973736299350350172 4
0, 55031811050977556866721522943961683298023290287356356115 60, 729491448815283993 2
50331811050977556866721522943961683298023290287356356115 60, 406756864289834115971
41995219449939828973736299350350172 40, 29454807414091435984137306883049956427877533 1
8228818460760, 20895290729654608433191422831565353705246455163503583952 40, 145204562
69759982131539802306681008507035672232 6520240760, 98836719147105760727287729986652 2
42778982306791048856724 0, 65891146098070507151525153324434828519321537860699237816 0,
43019839187996446817937910021738441760548772652853221 3840, 2750448734970264632622259
82106196594862524939911684530160, 17218256308350437131049919205022063255621479978211
1453840, 10553124834150267919030595641787716188929294180193927816 0, 6331874900490160
75141835738507262971335757650811635668 96, 37187201796529515524203051309156714189560 3
69968302412304, 21375320717690193962730887760381418392424464627449418096, 1202361790
37007341040361243652145478457387613529402976 79, 66176501640523420262524405420948286 5
9282574077974892521, 35633500883358764756743910611279846626906168112172498 19, 187687
90541616448612330762077697018452339890074350399 81, 96687708850751401942309986460863 4
28390841857958774786 9, 48707342052634164888231346562991351144258680325097073 1, 23990
18339905861852703931994893603862329158881683885 69, 11550829043991186698204117012450 6
85263066320541440931 1, 54356842559958525638607609470356165943841508430310264, 249962
12272097716169578681727244076309941715555544136, 11230182325145350742854190341225599
501568017133650264, 49283533944882474483029180634155508604003528428249 36, 2112151454
78067747784410774146380751160015121835354 4, 88380805554652461838866919678272796584 68
71786403256, 36099202268801709765170995361548043675435608177034 4, 143891925127391430
53249977172085583842656151511125 6, 55957970882874445207083244558110603832551700321 04

```
4, 21225437231435134388893644487559194557174782880396, 78505041814897072397277863173
16414425256426544804, 28304538885643162156841678559031970376774871215 96, 99448379868
47597514565995169389611213461441238 04, 34039378344243454580058104271065112207149899 5
396, 11346459448081151526686034757021704069049966513 2, 36819636619601087735603688946
626721813473401268, 11627253669347711916506428088408438467412653032, 357177073502838
20919987066676810235814927757 68, 10668925572162699755320812124242018490173226 32, 309
74300048214289612221712618767150455341624 8, 87363410392399278393445856104215039745 83
5352, 23927558260338655865720839569944246554591848, 63604901704697692807612358350484
70603119352, 16401263961588713239698784228741339291062 48, 41003159903971783099246960
5718533482276562, 99324424612105561544759718155421154091838, 23298321822592662584573
267221641999107962, 52885761192388252492589624981641347668 38, 11609069530036445669 10
503963011639339062, 24625299003107612025374326488125682949 8, 50437359403955349931489
584373269471102, 99666638343145302259829717623825900 98, 18984121589170533767377089 07
120493352, 34822944926880860750123654509310824 8, 61452255753319166029629978545842 63
2, 10421727583896232835434323846955768, 16965603043552006941404713239230 32, 26478108
79629503973514030389937 68, 39564990155383392707680913872632, 56521414507690561010972
73410376, 77074656146850765014962819232 4, 10015350798743319747707033007 6, 1237852345
9120956991548018324, 14522290091706150660475328 76, 16135877879673500733861476 4, 1693
8214348828536681954036, 16752080125215036278855 64, 15562041646374623865603 6, 1353221
0127282281622264, 1097206226536401212616, 82585414900589338584, 5741232051912627816,
366461620334848584, 21328454093562616, 1122550215450664, 52895036330136, 22039598470
89, 79936367511, 2472258789, 63391251, 1293699, 19701, 199, 1]
```

4. Add or double

[20 points] If you start with 1 RMB and, with each move, you can either double your money or add another 1 RMB, what is the smallest number of moves you have to make to get to exactly x RMB? Here x is an integer randomly selected from 1 to 100. Write a function Least_moves to print your results. For example, Least_moves(2) should print 1, and Least_moves(5) should print 3.

In [6]:
```python
def Least_moves(x):
    moves = 0
    while x > 1:
        if x % 2 == 0:
            x = x // 2
        else:
            x = x-1
        moves += 1
    print(moves)
Least_moves(3)
Least_moves(5)
```

```
2
3
```

5. Dynamic programming

Insert + or - operation anywhere between the digits 123456789 in a way that the expression evaluates to an integer number. You may join digits together to form a bigger number. However, the digits must stay in the original order.

5.1 [30 points] Write a function Find_expression, which should be able to print every possible solution that makes the expression evaluate to a random integer from 1 to 100. For example, Find_expression(50) should print lines include: [1 - 2 + 34 + 5 + 6 + 7 + 8 - 9 = 50] and [1 + 2 + 34 - 56 + 78 - 9 = 50]

In [7]:
```
def Find_expression(target):
    digits = "123456789"
    def search(expr, i):
        if i == 9:
            if eval(expr) == target:
                print(expr, "=", target)
            return
        for op in ["", "+", "-"]:
            search(expr + op + digits[i], i + 1)

    search(digits[0], 1)
Find_expression(50)
```

```
12+3+4-56+78+9 = 50
12-3+45+6+7-8-9 = 50
12-3-4-5+67-8-9 = 50
1+2+34-56+78-9 = 50
1+2+34-5-6+7+8+9 = 50
1+2+3+4-56+7+89 = 50
1+2+3-4+56-7+8-9 = 50
1+2-34+5-6-7+89 = 50
1+2-3+4+56+7-8-9 = 50
1-23+4+5-6+78-9 = 50
1-23-4-5-6+78+9 = 50
1-2+34+5+6+7+8-9 = 50
1-2+34-5-67+89 = 50
1-2+3-45+6+78+9 = 50
1-2-34-5-6+7+89 = 50
1-2-3+4+56-7-8+9 = 50
1-2-3-4-5-6+78-9 = 50
```

5.2 [5 points] Count the total number of suitable solutions for any integer i from 1 to 100, assign the count to a list called Total_solutions. Plot the list Total_solutions, so which number(s) yields the maximum and minimum of Total_solutions?

In [8]:
```
def count_solutions(target):
    digits = "123456789"
    count = 0
    def search(expr, i):
        nonlocal count
        if i == 9:
            if eval(expr) == target:
                count += 1
            return
        for op in ["", "+", "-"]:
            search(expr + op + digits[i], i + 1)
    search(digits[0], 1)
    return count
```

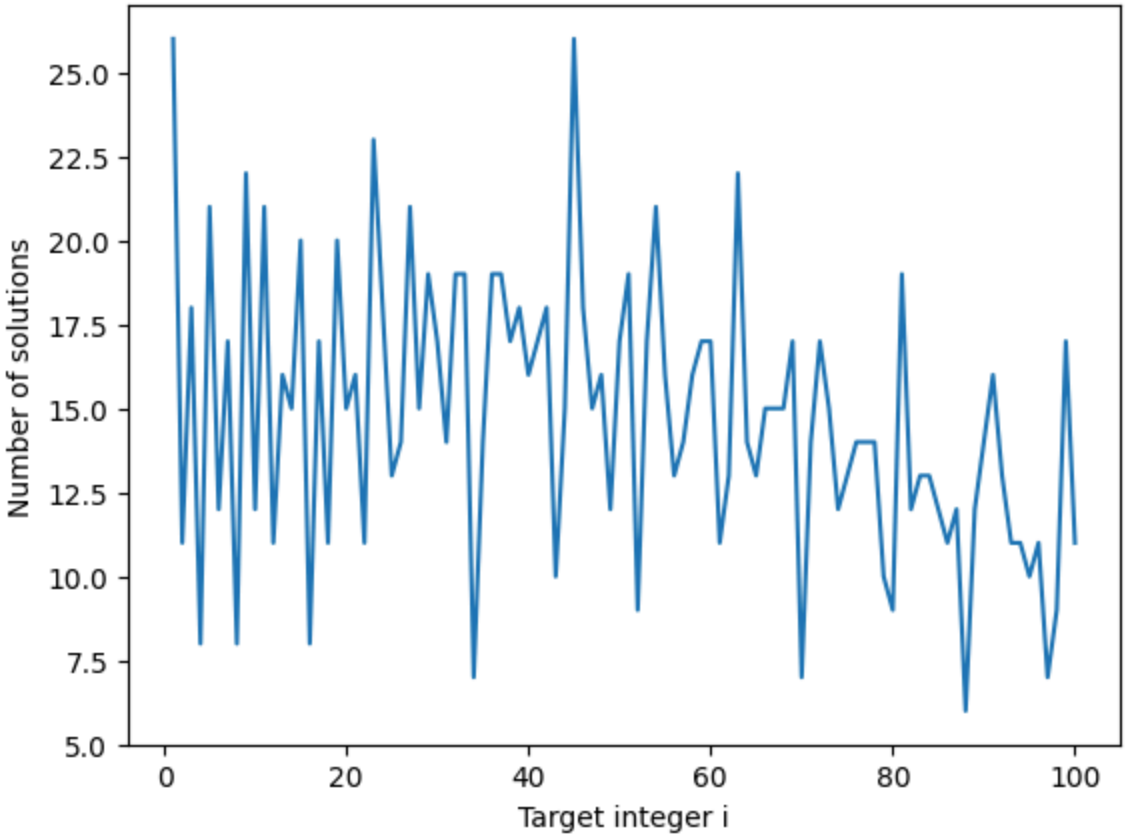
```python
Total_solutions = []
for i in range(1, 101):
    Total_solutions.append(count_solutions(i))

import matplotlib.pyplot as plt

plt.plot(range(1, 101), Total_solutions)
plt.xlabel("Target integer i")
plt.ylabel("Number of solutions")
plt.show()

max_count = max(Total_solutions)
min_count = min(Total_solutions)

for i in range(100):
    if Total_solutions[i] == max_count:
        print(f"最大值: i = {i + 1}, 共有 {max_count} 个解")
    if Total_solutions[i] == min_count:
        print(f"最小值: i = {i + 1}, 共有 {min_count} 个解")
```




```
最大值: i = 1, 共有 26 个解
最大值: i = 45, 共有 26 个解
最小值: i = 88, 共有 6 个解
```